



Visual Algorithm Race Simulator

Project implementation plan — RGPV B.Tech CSE Semester IV

CS402 Algorithms	BT401 Statistics	CS406 Java / Python	CS403 Software Engg.
------------------	------------------	---------------------	----------------------

CONCEPT OVERVIEW

Run multiple sorting and searching algorithms **simultaneously** on the same dataset and watch them compete in real-time. The app visualizes each swap, comparison, and pointer movement as it happens, displays live stats, and explains *why* one algorithm outperforms another — building genuine intuition for Big-O complexity that goes beyond memorizing formulas.

This stands out from typical student projects (library systems, student portals) because it is educational infrastructure — the kind of tool professors actually use in class — while demonstrating deep mastery of algorithms, not just implementation.

APP MODULES

Race Arena Side-by-side live visualization of algorithms on the same array	Stats Panel Live comparisons — swaps, comparisons, time elapsed, memory used	Dataset Designer Random, nearly-sorted, reversed, all-same — watch rankings flip
Complexity Graph Real-time $O(n)$, $O(n \log n)$, $O(n^2)$ curves vs actual performance	Insight Engine Explains WHY an algorithm wins/loses on each dataset type	Leaderboard Persistent rankings across dataset types, sizes, and scenarios

ALGORITHMS & UNIQUE FEATURES

Algorithms included	Unique features
■ Bubble, Selection, Insertion sort ■ Merge Sort, Quick Sort, Heap Sort ■ Binary Search vs Linear Search ■ BFS vs DFS on a random graph ■ Dijkstra vs Bellman-Ford (bonus)	■ Adjustable speed — slow-mo to instant ■ Step-through mode with explanation ■ Sound FX — each algorithm has a tone ■ Export race as GIF / shareable link ■ Quiz mode — predict the winner first

IMPLEMENTATION PHASES (~3 weeks total)

1	Core engine Implement all sorting algorithms as step-generators (yield each swap). Build the shared array state model. Unit test each algorithm with edge cases (empty, single, duplicates).	1 week
2	Visualization layer Canvas-based bar chart renderer. Color-code: compared bars = amber, swapped = coral, sorted = teal. Synchronize multiple canvases to the same array tick. Add speed slider.	1 week
3	Dataset designer + stats Input modes: random, nearly sorted, reversed, all duplicates, custom CSV upload. Live stats panel showing comparisons, swaps, memory allocations per algorithm in real-time.	4 days
4	Complexity graph + insight engine Plot actual step count against theoretical O curves using Chart.js. Write rule-based insight text explaining performance differences. Link to relevant BT401 statistical concepts.	4 days
5	Polish + extras Quiz mode (predict the winner), sound effects via Web Audio API, GIF export, responsive layout, leaderboard with localStorage. Final testing and presentation prep.	3 days

RECOMMENDED TECH STACK

Option A — Web app (recommended)		Option B — Java desktop app		
HTML + CSS + JS	Canvas API	Java Swing / JavaFX	JFreeChart	Timer
No framework needed — pure JS keeps it simple and fast to run in any browser.		Directly uses CS406 Java skills — good for college submission and demos.		

The web version runs in any browser with zero setup — ideal for demos. The Java version maps directly to CS406 coursework.